

Ghost of the Machine

- **Batas waktu:** 10 menit
- **Skor baseline:** 28.6
- **Lingkungan:** satu GPU (≈ 16 GB VRAM), tanpa internet
- **Ukuran solusi:** `solution.ipynb` ≤ 20 MB
- **Penyimpanan:** 5 GB
- **Model pralatih (pretrained):** hanya [bge-base-en-v1.5](#) — sebuah **encoder** teks (model embedding).

Tugas

Hal-hal aneh terjadi di National Archive of Kazakhstan. Para pustakawan mengatakan bahwa beberapa buku dahulu memiliki akhir cerita yang berbeda, tetapi tidak ada yang bisa membuktikannya — setiap salinan buku dari sebuah cerita memiliki akhir yang sama, dan setiap cerita tetap masuk akal. Anda diundang sebagai peneliti AI untuk menemukan lokasi perubahan tersebut.



Sebuah bacaan dimulai sebagai teks tulisan manusia dan, pada suatu titik, secara diam-diam beralih ke teks yang dihasilkan oleh sebuah *language model*. Dibaca secara keseluruhan, teks itu tampak seperti satu kesatuan yang koheren — tetapi di suatu tempat di tengah-tengah cerita penulisnya berubah dari seorang manusia menjadi sebuah mesin. Tugas Anda adalah **menemukan peralihan itu: indeks karakter tempat bagian manusia berakhir dan bagian mesin dimulai**.

Setiap sampel berupa satu string tunggal `text`. Terdapat tepat satu batas atau titik perubahan. Segala sesuatu sebelum batas tersebut ditulis oleh manusia; sedangkan segala sesuatu mulai dari titik tersebut dan seterusnya dihasilkan oleh mesin.

Dataset

Bacaan berbahasa Inggris dalam teks biasa (*plain text*), masing-masing dengan satu batas (titik perubahan).

- **Part A** (sebelum batas): sebuah kutipan teks tulisan manusia.
- **Part B** (mulai dari batas dan seterusnya): kelanjutan yang dihasilkan oleh sebuah *language model*, dikondisikan pada (*conditioned on*) Part A.
- Setiap sisi terdiri dari sedikitnya 180 kata; panjang total sekitar ~500–800 kata.
- Nilai indeks `boundary_char_index` adalah **indeks karakter pertama dari Part B**: `text[boundary_char_index:]` adalah bagian mesin sepenuhnya dan `text[:boundary_char_index]` adalah bagian manusia (*together with the single space that separates the two parts*).

Apa yang Anda dapatkan

Anda menerima **dua folder**:

Folder	Sampel	answers.jsonl?	Gunakan untuk
dataset/train/	1,221	☐ disertakan	melatih / melakukan fine-tune metode Anda
dataset/test_public/	380	☐ disertakan (salinan dev)	menjalankan pipeline Anda dan menilai sendiri secara lokal

Pada **waktu penilaian**, folder `dataset/test_public/` **diganti dengan set evaluasi tersembunyi**. Formatnya sama tetapi **tanpa** `answers.jsonl`. Notebook Anda dijalankan ulang dengan folder `dataset/test_public/`, dan `answers.jsonl` yang dihasilkan akan dinilai.

- Leaderboard publik menggunakan set tersembunyi **test_leaderboard_a** (380 sampel).
- Peringkat akhir menggunakan set tersembunyi **test_leaderboard_b** (380 sampel).

Ketiga set evaluasi tersebut berukuran sama dan diambil dari distribusi yang sama dengan `train`, sehingga skor `dataset/test_public/` lokal Anda merupakan estimasi yang wajar untuk skor leaderboard Anda.

Format pada disk

```
dataset/train/data.jsonl      # one JSON object per line: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": 1842}
dataset/test_public/data.jsonl      # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl  # dev copy only – ABSENT in the hidden grading set
```

- Id pada `answers.jsonl` cocok dengan id pada `data.jsonl`.
- `dataset/train/` (dengan jawaban) tersedia kapan pun Anda melatih atau melakukan fine-tune.

Output (format pengumpulan)

Anda mengumpulkan **satu notebook**, yang harus diberi nama `solution.ipynb`. Nama file yang tepat ini diwajibkan. Apa pun selain itu ditolak tanpa dijalankan.

Notebook Anda harus **membaca** `dataset/test_public/data.jsonl` dan menulis satu file `answers.jsonl` di root repositori — satu objek JSON per baris, yang memetakan setiap id sampel ke indeks karakter batas yang Anda prediksi:

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- `boundary_char_index` harus berupa **bilangan bulat dalam $[0, \text{len}(\text{text})]$** .
- Setiap id pada `dataset/test_public/data.jsonl` harus muncul tepat satu kali. Sampel yang tidak ada pada `answers.jsonl` (atau dengan nilai bukan bilangan bulat / di luar rentang) memperoleh skor 0 untuk sampel tersebut.

Penilaian

Untuk setiap sampel, misalkan p adalah indeks yang Anda prediksi dan t adalah batas sebenarnya. Skor per sampel meluruh secara eksponensial terhadap jarak karakter:

$$\text{score} = \exp\left(-\frac{|p - t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

Hal ini menghasilkan perilaku skor sebagai berikut:

- **≈1.0** — karakter batas tepat;
- **≈0.78** — selisih 25 karakter;
- **≈0.61** — selisih 50 karakter;
- **≈0.37** — selisih 100 karakter;
- **≈0.01** — selisih 500 karakter.

Skor akhir adalah rata-rata dari skor per sampel atas seluruh sampel dalam split tersebut (dilaporkan pada skala 0–100). Metrik ini memberi imbalan untuk hasil yang *dekat*, bukan hanya yang tepat.

Batasan

- **Lingkungan:** satu GPU (≈16 GB VRAM), tanpa internet pada waktu penilaian — model yang diizinkan (di bawah) sudah disediakan. **Anggaran waktu nyata (wall-clock): 10 menit** untuk seluruh proses — ini harus mencakup pelatihan / fine-tuning apa pun yang Anda lakukan pada waktu penilaian **ditambah** inferensi pada set evaluasi.
- **Model pralatih yang diizinkan** — daftar ini bersifat lengkap; tidak ada bobot pralatih lain yang boleh digunakan. Model ini **sudah disediakan di dalam lingkungan** (muat secara normal, misalnya `from_pretrained`; tidak ada internet pada waktu penilaian):
 - **bge-base-en-v1.5** — sebuah **encoder** teks berukuran 110M parameter (model embedding). Model ini menghasilkan embedding kalimat/bacaan; ini bukan generative language model. Anda boleh menggunakannya **apa adanya (frozen feature)** atau melakukan ***fine-tune* padanya di split train** (fine-tuning penuh masuk dalam anggaran 16 GB / 10 menit).
- **Solusi klasik / statistik tidak dibatasi:** Anda boleh membangun model berbasis fitur apa pun (misalnya, **classifier** atau **regressor** `scikit-learn`) di atas fitur *embedding* yang Anda hitung sendiri. Bobot *pretrained deep learning model* hanya dibatasi pada daftar di atas.